

Assignment: SDD, BDD, and TDD in AI-Assisted Software Development

Student Information

- Name: [吳國鋒]
 - Student ID: [1113306]
 - Course: AI-Assisted Software Development (CS351)
 - Date: 2026/05/22
 - Due Date: 2026/5/31 23:59:59
-

1. Introduction

在生成式 AI 全速發展的時代，AI 輔助軟體開發（AI-Assisted Software Development）已成為工程師的日常。透過大型語言模型（LLMs），AI 能夠在數秒內生成成百上千行的程式碼。然而，AI 雖然強大，卻缺乏「真正的自主理解能力」，它無法憑空猜測出開發者或業主心中真正的業務邏輯與隱性需求。

因此，在 AI 時代中，「清晰且精準的系統需求」比以往任何時候都更加重要。如果開發者給予模糊的 Prompt（提示詞），AI 就只能給出充滿 Bug 或不符合預期的程式碼。

為了有效駕馭 AI，**SDD（規格驅動）、BDD（行為驅動）** 與 **TDD（測試驅動）** 成為了現代工程師不可或缺的思維工具：

- **SDD** 幫助我們為 AI 建立嚴謹的框架與架構限制。
 - **BDD** 透過人類與 AI 都能看懂的自然語言情境，對齊雙方的行為預期。
 - **TDD** 則在程式碼落地前先建立防禦防線，自動化地檢驗 AI 生成的程式碼是否正確。這三者的結合，讓軟體開發從「碰運氣的盲盒」轉變為「精準可控的工程」。
-

2. Definition of SDD

規格驅動開發 (Specification-Driven Development, SDD) 是一種以「系統規格說明書」為核心的開發流程。在撰寫任何程式碼之前，開發團隊必須將系統的目標、功能、輸入輸出格式以及約束條件定義得清清楚楚。

在 AI 輔助開發的脈絡下，SDD 的核心價值在於為 AI 提供一份高質量的「系統結構藍圖」，其主要包含以下元素：

- **目標 (Goal)**：定義系統存在的核心理由與要解決的痛點。
- **功能需求 (Functional Requirements)**：系統必須具備的能力與核心動作。
- **輸入 (Input) 與 輸出 (Output)**：嚴格定義資料進出系統的格式、型態與邊界。
- **約束條件 (Constraints)**：系統運作時必須遵守的法規、效能或業務邏輯限制。
- **驗收標準 (Acceptance Criteria)**：評估系統是否建構成功的客觀依據。

藉由完整的 SDD 規格，我們可以將其直接轉化為 AI 的 System Prompt，確保 AI 在特定的限制內生成程式碼，避免產生幻覺（Hallucination）。

3. SDD: Student Grade Calculator

3.1 Goal

本系統旨在建立一個自動化的「學生學期成績計算器」，協助教學人員精準、快速地匯入學生的各項表現分數，並自動依據加權比重計算出最終學期總分與對應的等第級別，以降低人工結算成績的錯誤率。

3.2 Functional Requirements

1. **多欄位資料接收**：系統應能同時接收並獨立驗證四個不同的學生成績組件（作業、期中考、期末考、專案）。
2. **加權總分計算**：系統必須依據預設的加權公式（30%, 20%, 30%, 20%）精確計算出學期總分。
3. **四捨五入動態處理**：系統在計算出總分後，必須在判定等第與輸出前，將分數四捨五入至小數點後第一位。
4. **等第自動映射**：系統應依據法定的分數區間，自動將四捨五入後的總分轉換為 A、B、C、D 或 F 等第。
5. **例外與錯誤阻斷**：當輸入的分數超出合法的數值邊界時，系統應立即終止計算並拋出明確的錯誤提示。

3.3 Input

系統需接收四個浮點數（Float）或整數（Integer）輸入：

- **作業分數 (Assignment Score)**: 區間為 0.0 至 100.0。
- **期中考分數 (Midterm Exam Score)**: 區間為 0.0 至 100.0。
- **期末考分數 (Final Exam Score)**: 區間為 0.0 至 100.0。
- **專案分數 (Project Score)**: 區間為 0.0 至 100.0。

3.4 Output

系統結算後應輸出兩個欄位：

- **學期加權總分 (Final Score)**: 顯示至小數點後第一位（Float）。
- **等第文字 (Letter Grade)**: 顯示為單一英文字母字串（A, B, C, D, F）。

3.5 Grade Rules

總分判定前需先四捨五入至小數點後第一位，區間規則如下：

- **A**: $90.0 \leq \text{總分} \leq 100.0$
- **B**: $80.0 \leq \text{總分} < 90.0$
- **C**: $70.0 \leq \text{總分} < 80.0$
- **D**: $60.0 \leq \text{總分} < 70.0$
- **F**: $\text{總分} < 60.0$

3.6 Acceptance Criteria

- **AC 1 (精確加權驗證)**: 當輸入作業 80、期中 70、期末 85、專案 90 時，計算過程必須嚴格執行 $80 \times 0.3 + 70 \times 0.2 + 85 \times 0.3 + 90 \times 0.2$ ，且最終產出的總分必須精確為 81.5，等第必須為 B。
- **AC 2 (極值邊界攔截)**: 當任何一項輸入欄位小於 0.0（如 -5.0）或大於 100.0（如 105.5）時，系統不能產出任何等第，且必須回傳訊息 "Error: Invalid score range. All scores must be between 0 and 100."。

4. Definition of BDD

行為驅動開發 (Behavior-Driven Development, BDD) 是一種從「使用者故事與行為描述」出發的開發方法。它演變自 TDD，但將關注點從技術層面的測試，提升到了業務邏輯的行為描述。

BDD 的核心在於使用**無技術背景的人也能看懂的自然語言**，透過 **Given-When-Then** 語法來撰寫場景 (Scenario)：

- **Given (情境)**：系統目前所處的初始狀態或先決條件。
- **When (當)**：使用者觸發了某個核心動作或事件。
- **Then (則)**：系統應該產出的預期反應、狀態改變或輸出結果。

在 AI 輔助開發中，BDD 扮演著極佳的「溝通橋樑」。工程師可以將這些 Given-When-Then 的劇本直接當作 User Story 輸入給 AI，AI 能夠非常精準地理解使用者的業務行為，進而生成高度符合商業邏輯的應用層程式碼，避免了技術實現與真實需求脫節的問題。

5. BDD: Student Grade Calculator

Scenario 1: 學生學期表現優異獲得等第 B

- **Given** 學生的作業分數是 88
- **And** 期中考分數是 75
- **And** 期末考分數是 82
- **And** 專案分數是 85
- **When** 系統啟動學期總分與等第計算
- **Then** 計算出的學期總分應該為 83.0
- **And** 最終呈現的字母等第應該為 B

Scenario 2: 輸入分數超出上限觸發系統阻斷

- **Given** 學生的作業分數是 120 (超出合法範圍)
- **And** 期中考分數是 80
- **And** 期末考分數是 85
- **And** 專案分數是 90
- **When** 系統嘗試處理這組分數
- **Then** 系統應該拒絕執行加權計算
- **And** 系統必須顯示錯誤訊息 "Error: Invalid score range. All scores must be between 0 and 100."

6. Definition of TDD

測試驅動開發 (Test-Driven Development, TDD) 是一種「測試先行」的軟體開發控制流程。與傳統先寫程式碼再人工測試的方法相反，TDD 要求開發者在還沒動手寫任何一行業務邏輯之前，就必須先針對需求寫好自動化測試案例。

TDD 嚴格遵循一個被稱為 **紅-綠-重構 (Red-Green-Refactor)** 的敏捷循環：

1. **紅燈 (Red)**: 撰寫一個必定會失敗的測試案例（因為此時系統根本還沒實作該功能）。

2. **綠燈 (Green):** 用最快速、最簡單（甚至可以先 Hardcode）的方式寫出業務程式碼，目標是讓剛剛失敗的測試案例順利通過。
3. **重構 (Refactor):** 在確保所有測試依然維持綠燈的前提下，優化程式碼的結構、可讀性與效能，消除重複程式碼。

在 AI 時代，TDD 簡直是工程師的「防錯安全網」。當 AI 幫我們生成程式碼時，我們往往無法一眼看出代碼中隱藏的邊界 Bug。如果我們先用 TDD 定義好測試案例，就可以把 AI 生成的程式碼直接丟進測試套件中跑。如果全過（綠燈），代表 AI 代碼邏輯正確；如果有錯（紅燈），就能立刻把報錯回饋給 AI 修改。

7. TDD: Student Grade Calculator

Scenario 1: Normal Test Cases (常規常態測試案例)

Test Case 1: 常規及格等第 C 評定

- **Input**
 - Assignment: 72.0
 - Midterm: 68.0
 - Final Exam: 75.0
 - Project: 70.0
- **Expected Calculation**

$$FinalScore = 72.0 \times 0.30 + 68.0 \times 0.20 + 75.0 \times 0.30 + 70.0 \times 0.20$$

$$FinalScore = 21.6 + 13.6 + 22.5 + 14.0 = 71.7$$

- **Expected Output**
 - Final Score: 71.7
 - Letter Grade: C

Test Case 2: 常規低分不及格等第 F 評定

- **Input**
 - Assignment: 50.0
 - Midterm: 45.0
 - Final Exam: 55.0
 - Project: 40.0
- **Expected Calculation**

$$FinalScore = 50.0 \times 0.30 + 45.0 \times 0.20 + 55.0 \times 0.30 + 40.0 \times 0.20$$

$$FinalScore = 15.0 + 9.0 + 16.5 + 8.0 = 48.5$$

- **Expected Output**
 - Final Score: 48.5
 - Letter Grade: F

Scenario 2: Boundary Test Cases (臨界點邊界測試案例)

Test Case 1: 等第 A 的絕對下限臨界點 (90.0)

- **Input**

- Assignment: 90.0
- Midterm: 90.0
- Final Exam: 90.0
- Project: 90.0

- **Expected Calculation**

$$FinalScore = 90.0 \times 0.30 + 90.0 \times 0.20 + 90.0 \times 0.30 + 90.0 \times 0.20 = 90.0$$

- **Expected Output**

- Final Score: 90.0
- Letter Grade: A

Test Case 2: 四捨五入進位跳級邊界 (89.95 進位至 90.0)

- **Input**

- Assignment: 89.9
- Midterm: 90.0
- Final Exam: 90.0
- Project: 90.0

- **Expected Calculation**

$$FinalScore = 89.9 \times 0.30 + 90.0 \times 0.20 + 90.0 \times 0.30 + 90.0 \times 0.20$$

$$FinalScore = 26.97 + 18.0 + 27.0 + 18.0 = 89.97$$

四捨五入至小數點後第一位 => 90.0

- **Expected Output**

- Final Score: 90.0
- Letter Grade: A

Scenario 3: Invalid Input Test Cases (異常非合法的錯誤輸入測試)**Test Case 1: 單項分數低於合法下限 (負數)**

- **Input**

- Assignment: 85.0
- Midterm: -10.0
- Final Exam: 90.0
- Project: 80.0

- **Expected Calculation** 系統在輸入驗證階段即偵測到 Midterm (-10.0) 小於 0.0，引發例外狀況，不進行加權公式計算。

- **Expected Output**

- Final Score: null (或拋出 Exception)
- Letter Grade: "Error: Invalid score range. All scores must be between 0 and 100."

Test Case 2: 多項分數高於合法上限 (破百)

- **Input**
 - Assignment: 100.0
 - Midterm: 100.0
 - Final Exam: 105.0
 - Project: 100.0
- **Expected Calculation** 系統偵測到 Final Exam (105.0) 超過最大上限 100.0，立即阻斷程式流程。
- **Expected Output**
 - Final Score: null (或拋出 Exception)
 - Letter Grade: "Error: Invalid score range. All scores must be between 0 and 100."

8. Comparison of SDD, BDD, and TDD

比較項目	SDD (規格驅動開發)	BDD (行為驅動開發)	TDD (測試驅動開發)
完整名稱	Specification-Driven Development	Behavior-Driven Development	Test-Driven Development
主要關注點	系統宏觀結構、合約限制與全域規格	使用者故事、業務場景與外在行為表現	程式碼正確性、模組解耦與單元邊界
核心提問	整個系統應該具備什麼架構與限制？	系統在特定操作情境下應該如何表現？	我們該如何從代碼層面確保結果無誤？
典型撰寫格式	結構化規格書 (Markdown, Excel 矩陣)	自然語言劇本 (Given-When-Then)	單元測試框架代碼 (如 JUnit, PyTest)
主要參與對象	系統架構師、產品經理、AI 系統提示詞	產品經理、領域專家、測試工程師、QA	軟體工程師、AI 程式碼生成代理
AI 時代的新價值	作為 System Prompt 定義 AI 生成的骨架限制	作為 Few-Shot 範例 確保 AI 貼近商業邏輯	作為 自動化守門員 校驗 AI 是否有邏輯幻覺

9. Reflection

在這份作業中，透過將同一個「學生成績計算器」情境分別套用在 SDD、BDD 和 TDD 三種開發方法中，我深刻體會到現代軟體開發在 AI 介入後的思維轉變。

對我而言，**BDD（行為驅動開發）是最容易理解的方法**。因為它使用的是貼近人類日常對話的 **Given-When-Then** 結構。這種方式完全不需要去思考程式碼底層是用迴圈還是條件式，而是從一個使用者的視角出發，像是在講故事一樣去描述系統的行為。這種直覺的描述方式，不管理科還是文科背景的人都能在第一時間建立共識。

而在與 AI 編碼工具（如 GitHub Copilot、ChatGPT）協作時，我認為 **TDD（測試驅動開發）是當中最實用的工具**。AI 的強項是編寫代碼，但弱點是容易寫出「看起來很對，實際執行卻有邊界 Bug」的程式碼。如果我們能夠先實作 TDD，把正常、邊界、無效輸入的測試案例建置好，我們就能把 AI 當作一台「瘋狂產出程式碼的機器」，將其產出的結果丟入 TDD 安全網。只要紅燈變綠燈，我們就能確認 AI 沒有產生幻覺，這極大地提升了程式碼的交付信心。

此外，**SDD（規格驅動開發）**能有效解決 **AI 提示詞模糊（Vague Prompts）**的痛點。通常我們對 AI 下指令說「幫我寫一個學生成績計算器」，AI 根本不知道各科比重或有沒有四捨五入。而 SDD 規範了明確的輸入、輸出與限制（Constraints），將其作為 Prompt 輸入，AI 就能在我們設定好的框架內精準發揮，不會寫出偏離核心需求的系統。

如果未來我要主導一個大型的 AI 輔助軟體專案，我會將這三者**有機結合**：首先，利用 **SDD** 與產品經理或架構師定義好專案的範疇與 API 規格；接著，用 **BDD** 撰寫大量的使用者故事情境，確保開發方向完全符合業主需求；最後，將這些 BDD 情境轉化為 **TDD** 的自動化測試腳本，再交由 AI 協同生成業務程式碼並進行重構。這種組合拳能將 AI 的生產力發揮到極致，同時確保軟體的高質量。

10. References / AI Tool Usage

- **ChatGPT (OpenAI):** 用於協助釐清 SDD、BDD 與 TDD 三者現代 AI 開發環境下各自扮演的獨特角色，並輔助生成報告的結構大綱。
- **VS Code (Markdown Preview Enhanced):** 用於即時預覽本作業之 Markdown 排版效果。
- **報告審查：** 本報告內的所有成績計算數據、加權公式結果、BDD 劇本與 TDD 邊界測試案例，均由本人親自設計並透過計算機全數手動驗算確認，無直接抄襲作業說明範例之情事。
- **參考網站：**
 - Cucumber BDD Documentation: (<https://cucumber.io/docs/bdd/>)
 - Martin Fowler - Test Driven Development: (<https://martinfowler.com/bliki/TestDrivenDevelopment.html>)